

End-to-End QP Solver Pipeline

Large-Scale Convex Quadratic Programming (10^6+ Variables & Constraints)

Revision 6 — Presolve Safety, Honest GPU/ADMM Framing & Infeasibility Certification

Scope. This document specifies an end-to-end execution pipeline for a from-scratch convex QP solver core. It is the result of an iterative correction process: the original two-draft synthesis, an active-set/interior-point paradigm-mismatch correction, a canonical-augmentation rewrite that unified constraint and variable bounds into a single representation, a complementary-slackness verification fix, a pass that made the scaled/unscaled coordinate transform explicit and directionally correct end to end, and a final pass closing three remaining gaps: an under-specified presolve substitution, an overstated GPU/ADMM framing, and the absence of any infeasibility or unboundedness certification.

Canonical formulation

Problem. $\min_x \quad (1/2) x^T Q x + c^T x \quad \text{s.t.} \quad \tilde{l} \leq \tilde{A}x \leq \tilde{u}$

where $Q \in \mathbb{R}^{n \times n}$ is positive semi-definite ($Q \geq 0$), and direct variable bounds are folded into the constraint system by augmentation:

$$\tilde{A} = \begin{bmatrix} A & I_n \end{bmatrix} \in \mathbb{R}^{K \times n}, \quad K = m + n, \quad \tilde{l} = \begin{bmatrix} l & \hat{l} \end{bmatrix}, \quad \tilde{u} = \begin{bmatrix} u & \hat{u} \end{bmatrix} \in \mathbb{R}^K$$

Folding x 's own bounds into extra rows of $\tilde{A}$ means x is unconstrained in the reformulated problem — this is what keeps every later step (ADMM, IPM, the dual residual) working with a single, uniform K -dimensional constraint space instead of two separate bound systems with their own bookkeeping.

Scope limits — stated explicitly rather than left implicit

GPU role. GPU acceleration applies specifically to Step 3's warm-start (SpMV/PCG at $O(\text{nnz})$ cost per iteration). The high-precision solve — Step 5's repeated sparse LDL^T factorizations — is CPU-only, and is expected to dominate total FLOP count and memory pressure at 10^6+ scale. This is accurately described as a GPU-accelerated warm start with CPU-exact refinement, not a GPU-bound solver end to end.

Convexity. This pipeline solves convex QP ($Q \geq 0$) only, consistent with the sponsor's own phase-1 scope (LP/MILP/QP now; MIQP/NLP/MINLP as future extensions). Worth stating outright: genuine crude-blending and pooling problems are classically non-convex (bilinear concentration $\times$ flow-rate terms). This convex core solves the linearized or convexified relaxation of such problems, not the general pooling formulation — full non-convex blending requires the future MINLP-class extension, and benchmark results against blending case studies should be reported against that scope.

Step 1 — Ingestion, Condition Screening, Convexity Filter & Canonical Augmentation (CPU)

Before any reduction or solver iteration touches the system, verify well-posedness, screen for non-convexity, augment variable bounds into the constraint system, and equalize matrix norms while preserving $Q = Q^T$.

1. **Parsing & Matrix Augmentation.** Stack direct variable bounds into A to construct $\tilde{A} = [A; I_n] \in \mathbb{R}^{K \times n}$ in CSC and CSR formats. Store Q as upper-triangular CSC/CSR only (50% memory footprint, exploiting $Q = Q^T$).
2. **Convexity Screening Filter ($Q \geq 0$ fast diagnostic).** Execute an $O(nnz)$ necessary-condition check using Gershgorin disk bounds and diagonal inspection ($Q_{ii} \geq 0$).

Scope note

This is a fast, necessary-condition fail-fast filter, not a certificate of convexity — a matrix can be indefinite without producing an all-negative Gershgorin disk. The definitive certificate is Step 6's KKT inertia check during LDL^T factorization; this filter exists only to reject obviously bad input before VRAM is allocated.

3. **Condition Screening & Symmetric Ruiz Equilibration.** Compute $\kappa_{\text{est}} = \max(|Q_{ij}|, |\tilde{A}_{ij}|) / \min(|Q_{ij}|, |\tilde{A}_{ij}|)$; flag ill-conditioning above 10^8 . Compute diagonal scalings $D_x \in \mathbb{R}^{n \times n}$ and $D_y \in \mathbb{R}^{K \times K}$ over 10–20 passes:

$$\begin{aligned} (D_x)_{jj} &\approx 1/\sqrt{\max(\| \text{col}_j Q \|_{\infty}, \| \text{col}_j \tilde{A} \|_{\infty})} \\ (D_y)_{ii} &\approx 1/\sqrt{\| \text{row}_i \tilde{A} \|_{\infty}} \end{aligned}$$

Apply symmetrically: $Q_s = D_x Q D_x$, $A_s = D_y \tilde{A} D_x$, $c_s = D_x c$, $l_s = D_y \hat{l}$, $u_s = D_y \tilde{u}$

Note: $\hat{l}_s, \tilde{u}_s$ denote the bottom n rows of $\hat{l}_s, \tilde{u}_s$ — the scaled direct variable bounds. Bound vectors are scaled by the SAME D_y used for the rows of $\tilde{A}$, so that every quantity used in Steps 2–3 (matrices, bounds, and slacks alike) lives consistently in the scaled coordinate system. Steps 2 and 3 use **only** Q_s, A_s, c_s, l_s, u_s from this point on — never the unscaled $\tilde{A}, \hat{l}, \tilde{u}$. Those are reserved exclusively for Step 7's final, post-unscaling verification.

Step 2 — CPU Parallel Presolve (Algebraic Reduction)

Operating on scaled data (Q_s, A_s, c_s, l_s, u_s), apply reductions using multi-threaded OpenMP passes until no further reduction fires.

1. **Diagonal Zero Elimination & Free Variable Substitution.** If $|(Q_s)_{jj}| \leq 10^{-8}$ (numerically zero) and column $(A_s)_j$ has a singleton entry $(A_s)_{ij} x_j = b_i$, express $x_j = \alpha + \beta^T x_a$ in terms of the remaining variables and substitute it out of both $c_s^T x$ and $x^T Q_s x$.

Safety condition — this substitution is only valid because $Q_s \geq 0$

Since Q_s is symmetric positive semi-definite, a zero diagonal entry forces the entire row/column to vanish: for any i , taking $v = e_i + t \cdot e_j$ gives $v^T Q_s v = (Q_s)_{ii} + 2t(Q_s)_{ij}$, which is linear in the free

real parameter t ; PSD requires this to stay ≥ 0 for every t , which is only possible if $(Q_s)_{ij} = 0$ for all i . So eliminating x_j leaves $x^T Q_s x$ entirely unaffected by any cross term — the update is exactly $c_s \leftarrow c_s$ (restricted to x_a) + $\alpha \cdot (Q_s \text{ row/col } j)$, which is already zero) and a pure translation of the linear term, $c_a \leftarrow c_a + \alpha \cdot (A_s)_a$ -coupling from b_i . No rank-2 Hessian correction is needed in this branch, unlike the general affine-substitution case. This guarantee depends on the $Q_s \geq 0$ screen in Step 1 already having passed; if Step 6 later reveals Q_s was not in fact PSD, this substitution must be treated as unverified and the affected rows re-derived under the general (non-PSD-safe) rank-2 update rule instead.

2. Implied Bounds & Redundancy Detection

- *Feasibility Bound Tightening (FBT)*: compute row activity L_i, U_i using A_s . Flag infeasibility if $L_i > (u_s)_i$ or $U_i < (l_s)_i$. Drop redundant rows where the activity interval already lies inside $[(l_s)_i, (u_s)_i]$.
- *Dual Bound Tightening — DBT via interval arithmetic*: x 's value is unknown during presolve, so bound the vector $W = Q_s x$ using interval arithmetic over the known scaled variable bounds $x \in [\hat{l}_s, \hat{u}_s]$, not a direct pointwise substitution:

$$\begin{aligned} W_i^{\min} &= \sum_{(Q_s)_{ij} > 0} (Q_s)_{ij} (\hat{l}_s)_j + \sum_{(Q_s)_{ij} < 0} (Q_s)_{ij} (\hat{u}_s)_j \\ W_i^{\max} &= \sum_{(Q_s)_{ij} > 0} (Q_s)_{ij} (\hat{u}_s)_j + \sum_{(Q_s)_{ij} < 0} (Q_s)_{ij} (\hat{l}_s)_j \end{aligned}$$

Substitute the interval $[W_i^{\min}, W_i^{\max}]$ into $A_s^T y + c_s + W = 0$ to derive rigorous bounds on the dual multipliers $y \in \mathbb{R}^k$, catching early unboundedness.

3. **Duplicate Row/Column Merging**. Hash scaled column/row vectors (xxHash64 / MurmurHash3) to detect and merge linearly dependent rows or parallel columns.
4. **Presolve Transaction Log (M_presolve)**. Record every substitution, fix, and shift in a strict LIFO stack for exact post-solve restoration.

Step 3 — GPU-Accelerated Warm-Start: Native Matrix-Free ADMM Engine (GPU)

Transfer Q_s, A_s, c_s, l_s, u_s into VRAM in dual CSC(A_s)/CSR(A_s^T) and upper-triangular Q_s layouts. Execute a native CUDA SpMV/AXPY Matrix-Free ADMM loop — hand-written kernels, not vendor solver libraries — so the numerical critical path stays fully auditable, matching the same sovereignty rationale established for the LP pipeline. This step's purpose is twofold: (a) it delivers a cheap, GPU-parallel approximate point at $O(nnz)$ cost per iteration, entirely bypassing any CPU factorization; and (b) handing Step 5 a point already close to the central path reduces the number of expensive Newton/IPM iterations Step 5 needs to reach convergence. ADMM does not alter the conditioning of Q_s or A_s themselves — the KKT system Step 5 factors is exactly as conditioned as it would be without this step; what a good warm start buys is fewer factorizations of that system, not a better-conditioned one.

Notation

Every iterate below operates in the scaled coordinate system established in Step 1, and is labeled accordingly (x_s, z_s, y_s) — matching the convention Step 5 already uses for its own Newton directions $(\Delta x_s, \Delta y_s)$. Only the converged output, transferred to CPU at termination, is handed to

Step 4; un-scaling back to physical units happens once, in Step 7.

1. **Splitting formulation**

$$\begin{cases} \min_{x_s} (1/2) x_s^T Q_s x_s + c_s^T x_s + I_{[l_s, u_s]}(z_s) & \text{s.t.} \\ A_s x_s = z_s \end{cases}$$

2. **ADMM execution loop** (in-VRAM native SpMV/PCG kernels):

- *Primal update (Matrix-Free PCG solve):* $(Q_s + \sigma I + \rho A_s^T A_s) x_s^{(k+1)} = \sigma x_s^{(k)} - c_s + A_s^T (\rho z_s^{(k)} - y_s^{(k)})$, solved with Jacobi-preconditioned matrix-free PCG ($A_s^T A_s$ is never formed; applied as two sequential SpMVs).
- *Slack update & unified box projection:* $z_s^{(k+1)} = \text{proj}_{[l_s, u_s]}(A_s x_s^{(k+1)} + y_s^{(k)}/\rho)$ — covers original constraints and variable bounds in a single projection, since both now live in the same augmented, scaled row space.
- *Dual update:* $y_s^{(k+1)} = y_s^{(k)} + \rho(A_s x_s^{(k+1)} - z_s^{(k+1)})$

3. **Periodic penalty adjustment (p control).** Adjust ρ periodically (every 20–50 iterations, not every iteration) based on the primal/dual residual ratio, preserving the PCG Jacobi preconditioner's stability rather than destabilizing it with continuous re-tuning.

4. **Termination & PCIe transfer.** Stop at relative residuals $\epsilon \approx 10^{-3}$ to 10^{-4} ; transfer the converged scaled-space point $(\bar{x}_s, \bar{y}_s)$ to host CPU RAM to seed Step 4.

3.5 — Infeasibility & Unboundedness Certification (Cold-Start Path)

Before handing the converged point to Step 4, screen for primal infeasibility and dual infeasibility/unboundedness using ADMM's own iterate differences, following Banjac, Goulart, Stellato & Boyd (2019) (the certification theory built for exactly Stellato et al.'s OSQP splitting used above). This is a genuine Farkas-type certificate, not a heuristic, and it is the reason the cold-start (Step 3) path has stronger infeasibility guarantees than the hot-start path, which is instead covered by Step 5.4.

5. **Primal infeasibility certificate.** Track $\delta y_s = y_s^{(k+1)} - y_s^{(k)}$ across ADMM iterations. If δy_s converges in direction to a nonzero vector satisfying $A_s^T \delta y_s \rightarrow 0$ (i.e. $\delta y_s \in \text{null}(A_s^T)$ in the limit) together with a strictly negative signed projection onto the bound set $-l_s^T[\delta y_s]_+ - u_s^T[\delta y_s]_- < 0$ — this is a certificate that the original constraint system $l_s \leq A_s x_s \leq u_s$ is infeasible; report INFEASIBLE and terminate without proceeding to Step 4.
6. **Dual infeasibility / unboundedness certificate.** Track $\delta x_s = x_s^{(k+1)} - x_s^{(k)}$. If δx_s converges in direction to a nonzero vector satisfying $Q_s \delta x_s \rightarrow 0$, $A_s \delta x_s$ falling within the recession cone of $[l_s, u_s]$ (i.e. each component tends to 0, $+\infty$, or $-\infty$ consistently with that row's bound being one-sided or absent), and $c_s^T \delta x_s < 0$, this is a certificate that the objective is unbounded below on the feasible region; report UNBOUNDED and terminate without proceeding to Step 4.

Scope note — why an approximate certificate, not the Homogeneous Self-Dual embedding

A Homogeneous Self-Dual (HSD) embedding, as used in MOSEK and SeDuMi, would detect infeasibility/unboundedness as an intrinsic property of a single, larger self-dual problem, with no separate detection logic required. That is a more rigorous — and more invasive — architectural choice; it is not implemented here. This step instead reuses the ADMM iterates already being computed for the warm-start and applies a published, peer-reviewed certificate test to them, which detects the great majority of practically-occurring infeasible/unbounded instances without an HSD

rewrite of the whole pipeline. Genuinely pathological instances that evade this iterate-based test are still caught downstream by Step 5.4's divergence monitor during the exact IPM solve.

Step 4 — Interior-Point Initialization & Centering (CPU)

Map ADMM's approximate scaled-space point $(\bar{x}_s, \bar{y}_s) \in \mathbb{R}^n \times \mathbb{R}^k$ into a strictly feasible starting point $(x_s^0, y_l^0, y_u^0, s_l^0, s_u^0)$ in the relative interior of the scaled domain — not an active-set basis. Mehrotra Predictor-Corrector IPM (Step 5) is a barrier method: it consumes a strictly-interior continuous point, never a discrete active set. Handing it a purified vertex/basis (as an LP-style crossover would produce) breaks the barrier term at the first iteration; this step exists specifically to avoid that category error.

1. Paired slack computation & interior shift

- Compute scaled lower/upper primal slacks: $s_l = A_s \bar{x}_s - l_s \in \mathbb{R}^k, \quad s_u = u_s - A_s \bar{x}_s \in \mathbb{R}^k$
- Split ADMM's single dual vector into positive/negative parts: $y_l = \max(\bar{y}_s, 0), \quad y_u = \max(-\bar{y}_s, 0)$ — valid because at most one side is active per row, matching the standard single-multiplier range-constraint convention.
- Shift components away from the boundary by thresholds $\epsilon_p, \epsilon_d > 0$: $s_l^0 = \max(s_l, \epsilon_p), \quad s_u^0 = \max(s_u, \epsilon_p), \quad y_l^0 = \max(y_l, \epsilon_d), \quad y_u^0 = \max(y_u, \epsilon_d)$

2. Unified complementarity gap centering. Compute the initial gap across all $2K$ primal-dual pairs ($K = m+n$) — never a bare inner product of differently-scoped vectors:

$$\mu_0 = ((s_l^0)^T y_l^0 + (s_u^0)^T y_u^0) / (2K)$$

Step 5 — High-Precision CPU Primal-Dual Interior Point Method (IPM)

Drive accuracy from 10^{-4} to machine precision (10^{-8} to 10^{-12}) using Mehrotra Predictor-Corrector IPM on parallel multi-core CPU architectures, working entirely in the scaled coordinate system (x_s, y_s) .

1. Symmetric augmented KKT system. Solve, at each iteration:

$$\begin{bmatrix} -(Q_s + \theta_x) & A_s^T \\ A_s & \theta_y \end{bmatrix} \begin{bmatrix} \Delta x_s \\ \Delta y_s \end{bmatrix} = \begin{bmatrix} -(Q_s x_s + c_s + A_s^T y_s) \\ r_{pri} \end{bmatrix}$$

where $\theta_y = (S_l^{-1} Y_l + S_u^{-1} Y_u)^{-1} \in \mathbb{R}^{k \times k}$ is the standard combined two-sided-barrier diagonal term (correctly accounting for both the lower- and upper-bound slack pairs on every row), and θ_x provides free-variable regularization for the now-unconstrained x block.

2. Parallel direct factorization (LDL^T). Compute sparse symmetric indefinite LDL^T factorization using Nested Dissection ordering (METIS/OpenMP elimination trees); execute multifrontal parallel solves across CPU cores for both the predictor and corrector steps.

3. **Mehrotra predictor-corrector steps.** Solve the factorized system twice per iteration: first for the affine-scaling predictor direction $(\Delta x_s^{\text{aff}}, \Delta y_s^{\text{aff}})$, adjust the centering parameter σ , then solve for the combined corrector direction $(\Delta x_s, \Delta y_s)$.
4. **Infeasibility/Unboundedness Divergence Monitor.** Step 3.5's ADMM-based certificate only covers the cold-start path — the hot-start path (System Architecture Integration §1) bypasses Step 3 entirely and therefore never produces ADMM iterates to certify against. This monitor closes that gap and also serves as a backup on the cold-start path. The **sole primary, sufficient trigger** is the standard, well-established infeasible-start IPM divergence signature: sustained growth of $\|x_s\|$ and/or $\|y_s\|$ across iterations combined with a collapsing step length $\alpha_k \rightarrow 0$ before the duality gap has reached its target tolerance. When both conditions hold for several consecutive iterations, terminate and report INFEASIBLE_OR_UNBOUNDED (the sign pattern of the diverging y_s vs. x_s block distinguishes which, following the same interpretation as Step 3.5's two certificates). A shrinking duality gap combined with stalled primal/dual residuals is **not** used as an independent trigger: that pattern is equally consistent with slow-but-genuine convergence on a hard, well-posed instance, and treating it as sufficient on its own risks false positives on legitimately difficult feasible problems. It is retained only as corroborating evidence alongside the norm-divergence/step-collapse trigger, never as a standalone termination condition. As in Step 3.5, this is a heuristic monitor, not a Farkas certificate — an HSD embedding would detect this structurally instead of by iteration-count heuristics, but is not implemented here (see the Step 3.5 scope note).

Step 6 — In-Loop Inertia Control, Regularization & Stability (inside Step 5)

Fired reactively *inside* Step 5's LDL^T factorization loop whenever Q_s exhibits zero-eigenvalues or constraint rows become linearly dependent — this is a conditional branch, not a separate pipeline stage, mirroring how anti-degeneracy is scoped inside the LP pipeline's Simplex loop.

1. **Inertia check — the rigorous convexity/rank certificate.** Track matrix inertia (n_+, n_-, n_0) during LDL^T factorization. Target inertia is strictly $(n, K, 0)$, where $K = m+n$; any violation certifies local non-convexity or rank deficiency that Step 1's fast Gershgorin filter could not have caught.
2. **Dynamic regularization (δ -perturbation).** If $n_0 > 0$ or $n_+ \neq n$, inject dynamic diagonal perturbations into the KKT matrix's primal and dual blocks, restoring the correct target inertia without corrupting the solution beyond 10^{-8} accuracy.
3. **Iterative refinement.** If ill-conditioning causes residual drift ($\|r_x\| > 10^{-7}$), run:

$$\begin{array}{l} \left| \begin{array}{l} r^{\wedge}(k) = \text{rhs} - K_{\text{kkt}} x^{\wedge}(k) \\ x^{\wedge}(k) + \Delta x \end{array} \right. \Rightarrow K_{\text{kkt}} \Delta x = r^{\wedge}(k) \Rightarrow x^{\wedge}(k+1) = \end{array}$$

Step 7 — Un-Presolve, Feasibility Projection, Un-Scale & Verify (CPU)

This step applies only on the normal-convergence branch of Step 5. If Step 3.5 or Step 5.4 has already reported INFEASIBLE, UNBOUNDED, or INFEASIBLE_OR_UNBOUNDED, the pipeline terminates at that

point and Steps 7.1–7.4 below do not execute — there is no converged (x_s, y_s) pair to un-presolve, project, or un-scale.

On the normal-convergence branch, reversal proceeds in strict Last-In, First-Out order: undo presolve first, project onto physical bounds, then invert scaling — because scaling was applied first (Step 1) and presolve operated on the already-scaled system (Step 2).

1. **Un-presolving.** Replay the transaction tree M_{presolve} in reverse chronological order: $x_{\text{scaled}} \rightarrow x_{\text{scaled_orig}}$.

2. **Feasibility bound projection.** Eliminate variable-substitution roundoff before un-scaling, projecting onto the **scaled** bounds (still pre-unscaling at this point):

| $x_{\text{scaled_orig}} \leftarrow \text{proj}_{[l_s, u_s]}(x_{\text{scaled_orig}})$

3. **Un-scaling.** Invert Step 1's Ruiz scaling. The two directions are not symmetric — this is worth stating explicitly, since getting it backwards silently reintroduces exactly the scaled/unscaled mixing bug this revision exists to eliminate:

| $x_{\text{final}} = D_x \cdot x_{\text{scaled_orig}}$ $y_{\text{final}} = D_y \cdot y_s$
 (duals un-scale by MULTIPLYING by D_y) $s_{l,\text{final}} = D_y^{-1} \cdot s_l$
 $s_{u,\text{final}} = D_y^{-1} \cdot s_u$ (slacks un-scale by DIVIDING by D_y)
 $y_{l,\text{final}} = D_y \cdot y_l$ $y_{u,\text{final}} = D_y \cdot y_u$ (split
 parts of y follow y 's own transform)

4. **Final unscaled KKT verification gate** — against the original, unscaled $\tilde{A}, \tilde{l}, \tilde{u}, Q, c$ only:
 - *Primal residual:* $\|\tilde{A} x_{\text{final}} - z\|_{\text{inf}} / (1 + \|\tilde{u}\|_{\text{inf}}) \leq 10^{-8}$ where $z = \text{proj}_{[\tilde{l}, \tilde{u}]}(\tilde{A} x_{\text{final}})$
 - *Dual residual:* $\|Q x_{\text{final}} + \tilde{A}^T y_{\text{final}} + c\|_{\text{inf}} / (1 + \|c\|_{\text{inf}}) \leq 10^{-8}$
 - *Complementary slackness* (worst case across both bound sides, using the unscaled slacks/duals from step 3 above): $\max(\|S_{l,\text{final}} y_{l,\text{final}}\|_{\text{inf}}, \|S_{u,\text{final}} y_{u,\text{final}}\|_{\text{inf}}) \leq 10^{-8}$

System Architecture Integration

1. Parametric Hot-Starting Engine (Sequential Solve Path)

When re-solving sequential instances $QP(c+\Delta c, \tilde{l}+\Delta\tilde{l}, \tilde{u}+\Delta\tilde{u})$ — as in a scheduling/dispatch model re-solved every few minutes with minor data perturbations:

- Bypass Step 3 (GPU ADMM) entirely.
- Seed Step 5 (IPM) with the previous converged iterate $(x_s^*, y_l^*, y_u^*, s_l^*, s_u^*)$ pushed back into the interior: $s_{l,0} = \max(s_l^*, \delta)$, $s_{u,0} = \max(s_u^*, \delta)$. Convergence is typically fast for small perturbations, but no fixed iteration count is asserted here — warm-starting an interior-point method is a harder, less uniformly reliable problem than warm-starting an active-set method, since the previous point sits near the old boundary and must be pulled back into the interior before Newton steps behave well.

- Because this path bypasses Step 3, it has no ADMM iterates and therefore no Step 3.5 certificate; infeasibility or unboundedness introduced by the perturbation $(\Delta c, \Delta \tilde{l}, \Delta \tilde{u})$ is caught exclusively by Step 5.4's divergence monitor during the IPM re-solve. If Step 5.4 fires, fall back to a full cold start from Step 1 for that instance rather than trusting the hot-started result.

2. API / CLI Interface Layer

- `QPSolver::Status status = QPSolver::solve(QPModel& model, QPSolution& sol, QPOptions& opts);`
- Data ingestion formats: QPS, MPS (with QUADOBJ/QMATRIX extensions), and JSON specifications.

Consolidated Pipeline Summary

#	Phase	Method(s)	Hardware	Convergence / Scope Target
1	Ingestion & Scaling	Canonical augmentation ($\tilde{A} \in \mathbb{R}^{K \times n}$), Gershgorin filter, symmetric Ruiz scaling (Q_s, A_s, c_s, l_s, u_s)	CPU	Initial setup
2	Parallel Presolve	Singletons, FBT on $[l_s, u_s]$, DBT (interval arithmetic) on $[\hat{l}_s, \hat{u}_s]$, hash merging	CPU (OpenMP)	Reduced system
3	Warm-Start	Native CUDA Matrix-Free ADMM + PCG, unified box projection ($A_s x_s - z_s = 0, z_s \in [l_s, u_s]$), periodic ρ + infeasibility certification (3.5)	GPU	$\epsilon \approx 10^{-3} - 10^{-4}$ (bypassed on hot-start)
4	IPM Initialization	Paired slacks ($s_l, s_u, y_l, y_u \in \mathbb{R}^K$), interior shift, centering μ_0 over $2K$ pairs	CPU	Strict relative-interior point
5	Exact IPM Solve	Mehrotra Predictor-Corrector IPM, parallel multifrontal LDL^T factorization + divergence monitor (5.4, heuristic)	CPU (multi-core)	$10^{-8} - 10^{-12}$
6	Inertia & Stability	Inertia check ($n, K, 0$), dynamic δ -regularization, iterative refinement	CPU	Reactively, inside Step 5's loop
7	Post-Solve	LIFO un-presolving, scaled bound projection, direction-correct un-scaling, unscaled KKT gate	CPU	Final solution certificate

Key structural notes

Canonical augmentation is load-bearing, not cosmetic. Folding variable bounds into $\tilde{A}$ eliminates the need for a separate dual-slack object for x , resolves the earlier active-set/IPM category error at its root, and lets one projection step (Step 3) and one inertia check (Step 6) cover both constraint types uniformly.

Scaled vs. unscaled is enforced end-to-end, in one direction only. Every quantity from Step 1

through Step 6 — matrices, bounds, and slacks alike — lives in the scaled coordinate system. Only Step 7 crosses back, and the two un-scaling directions (multiply by D_y for duals, divide by D_y for slacks) are not interchangeable.

Scalability at 10^6 + scale is structure-dependent, not a flat guarantee. Step 5's Nested Dissection ordering controls fill-in during LDL^T factorization, but how well it controls fill-in depends on the sparsity structure of the underlying problem: mesh-like, network-flow, or staircase-structured instances (typical of refinery scheduling time-indexed formulations) order well and stay sparse through factorization, while densely-coupled instances (many variables interacting through a small number of global constraints, as pooling/blending relaxations can produce) can suffer substantial fill-in regardless of ordering heuristic. Benchmark scalability claims should report which structural regime the tested instances fall into, not a single blanket number.